Monolith vs Microservices

independence costs a distributed system

Watch the 4-min video

In one line: a monolith is one deployable with fast in-process calls and real transactions; microservices buy independent deployment, scaling and team autonomy at the price of network failures, eventual consistency and a lot of operations work, so they pay off only when that independence is worth more than the overhead.

1 Forty services and a pager that never stops

Many teams start with microservices because big tech companies talk about them. Months later they have forty services, a message broker, and every release still needs five teams. Microservices solve real problems, but mostly problems of **people and deployment**, not code. Without those problems you pay the cost and get no benefit.

2 What each style gives you

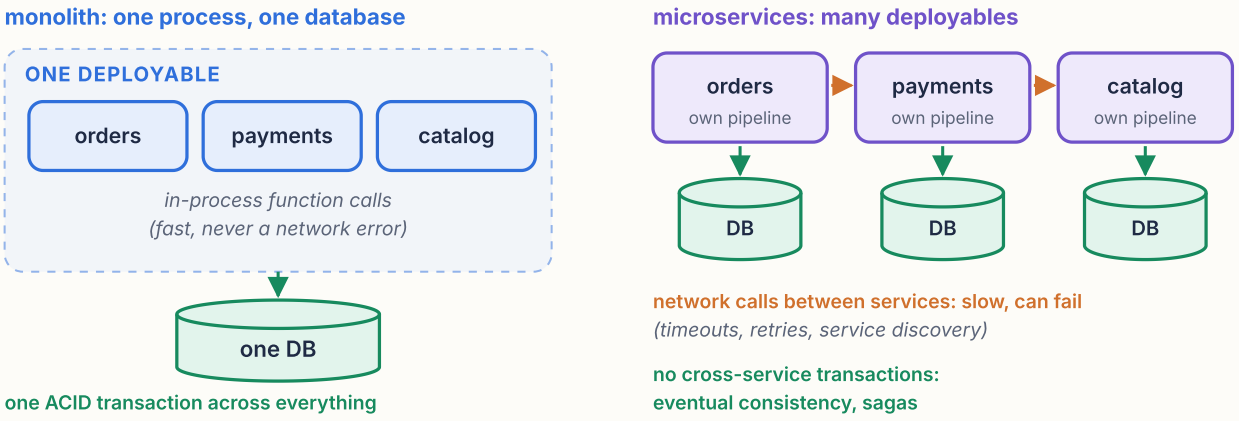


Figure 1. A monolith keeps calls and transactions local; microservices split code **and data**, so every hop is a network call.

A **monolith** is one deployable unit (often several identical copies behind a load balancer) and usually one database. Calls can't time out, one transaction keeps orders and stock consistent, and one stack trace shows the whole request.

Microservices split the system into services that each own their data and are deployed on their own, ideally by one team each. The big win is **independent deployment**: the payments team can ship ten times a day without a release train. Services can also be **scaled separately**, use different technology, and fail separately, so a bug in recommendations need not take down checkout.

3 What you pay

- **The network:** every call can be slow, fail, or succeed without you knowing. You now need timeouts, retries, circuit breakers and idempotency.
- **Data:** each service owns its database (lesson 23), so there are no joins or transactions across services. You accept eventual consistency and stale copies.

- **Operations and testing:** service discovery, a gateway, central logs, metrics and tracing, a pipeline per service, and contract tests for features that span services.

	Monolith	Microservices
Calls between parts	in-process, ~ns, can't fail on the network	HTTP/gRPC or messages, ~ms, can time out
Consistency	one ACID transaction	eventual; sagas for multi-step changes (lesson 34)
Deployment	all or nothing, one pipeline	per service, many pipelines
Scaling	scale the whole app	scale each service separately
Testing & debugging	one process, one stack trace	contract tests, distributed tracing

4 The distributed monolith trap and Conway's Law

A **distributed monolith** has separate services but keeps the coupling: they **share a database**, must be **deployed in lock-step**, or call each other in **long synchronous chains** (A → B → C → D on every request). You get the costs of microservices without the independence.

Conway's Law (Melvin Conway, 1968): a system's design mirrors the communication structure of the organisation that builds it. So service boundaries should match team boundaries (lesson 22). If every feature needs three teams, the boundaries are wrong.

WORKED EXAMPLE: SPLITTING OUT PAYMENTS FOR COMPLIANCE

An online store runs as a monolith. Card data (under PCI DSS rules) flows through the shared codebase and database, so the **whole** application is in audit scope. The team extracts one **payments service** with its own database (a card vault), network zone and pipeline. The monolith sends only an amount and order ID through a narrow API and gets back a payment token; it never sees card numbers. Audit scope shrinks to one small service, and the rest stays a monolith: one extraction for a real need, not a rewrite.

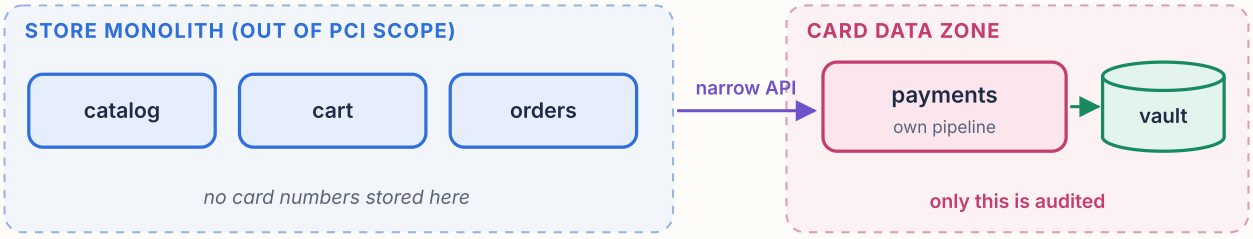


Figure 2. Only the payments service and its vault handle card data, so only they are in compliance scope.

WHEN NOT TO USE MICROSERVICES

- The team is small: one or two teams share one codebase easily.
- The domain is new or changing fast, so you don't yet know where the stable boundaries are.
- You lack strong automation: CI/CD, infrastructure as code, monitoring and tracing.
- No concrete pain yet. Start with a modular monolith (lesson 20) and split when a clear boundary, scaling need or team bottleneck appears (Fowler's "MonolithFirst").

COMMON MISTAKES

- Starting with microservices before the domain boundaries are known.
- Splitting the code but keeping one shared database: a distributed monolith.
- Splitting by technical layer instead of business capability.
- Adding services without tracing, per-service pipelines or on-call owners.

5 Key terms

Monolith: one deployable unit, usually with one database

Independent deployment: release one service without releasing others

Conway's Law: systems mirror the communication structure of the teams

Microservice: small, independently deployable service owning its data

Distributed monolith: separate services that are still tightly coupled

Eventual consistency: copies agree after a delay, not instantly

6 Interview questions

Q1. What is the main benefit of microservices? BASIC

Independent deployment: each team can build, test and release its service on its own schedule. Separate scaling and fault isolation follow from that.

Q2. What does a monolith give you that microservices don't? BASIC

Fast in-process calls that can't fail on the network, one ACID transaction across all data, and one thing to build, test, deploy and debug.

Q3. When should you NOT use microservices? SCENARIO

With a small team, an unclear or fast-changing domain, or weak automation and observability. Then the network, consistency and operations costs outweigh the benefits; start with a modular monolith.

Q4. Your store must pass card-data audits, and the whole monolith is in scope. What do you do? SCENARIO

Extract just payments into its own service, database, network zone and pipeline behind a narrow API, so only that service handles card data and the audit scope shrinks.

Q5. You have 30 services but every release still needs 5 teams. What's wrong? TRICKY

It's a distributed monolith: shared databases, lock-step deploys or boundaries that don't match teams. Fix the boundaries and data ownership, or merge services back.

Q6. How does Conway's Law affect service design? TRICKY

Systems mirror team communication, so service boundaries should match team boundaries; otherwise every feature needs cross-team coordination and the independence is lost.

Next: SD-L-002 · Modular Monolith backendroadmap.com